This project implements a biologically inspired reinforcement learning (RL) model to simulate decision making in the basal ganglia-thalamo-cortical (BG-TC) loop. This circuit, critical for action selection and motor control, is modulated by dopamine (DA) and serotonin (5-HT)—neuromodulators implicated in reward processing, impulsivity and behavioural inhibition.

The basal ganglia comprises subcortical structures such as the striatum, globus pallidus, subthalamic nucleus (STN) and substantia nigra (SN). These form two key pathways:

- Direct Pathway - facilitates action execution.
- Indirect Pathway - suppresses competing actions.

The cortex provides sensory/contextual input and the thalamus relays selected actions back to the cortex. The dopaminergic midbrain (SNc/VTA) evaluates action outcomes via prediction errors while serotonin is hypothesised to modulate behavioural inhibition and aversive sensitivity.

We map these biological components to an actor-critic RL architecture:

- Actor: Striatum and motor cortex - select action
- Critic: Dopaminergic structures (SNc/VTA) - evalurates actions using temporal difference (TD) error.
- Cortex: Encodes state
- Thalamus: Executes chosen actions

We implement a discrete-state, discrete-action actor-critic model. Each episode simulates goal directed behaviour:

- State: Represents context requiring motor/cognitive choice.
- Action: Candidate motor or cognitive output.
- Critic: Computes TD error from reward.

- Actor: Uses softmax policy, updated via TD error

The Temporal Difference error is given by:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

r_t : Reward at time t

γ : Discount factor

$V(s)$: Value function estimate for state s

Dopamine encodes this TD error, adjusting:

- Critic: $V(s_t) \leftarrow V(s_t) + \alpha_c \delta_t$
- Actor: $\pi(a|s_t) \leftarrow \pi(a|s_t) + \alpha_a \delta_t$

Serotonin modulates the softmax temperature parameter controlling the exploration-exploitation trade off:

- Low 5-HT → High temperature → More exploration
- High 5-HT → Low temperature → Greater exploitation (inhibition of impulsive actions)

```
class SimpleEnvironment:
    def __init__(self):
        self.states = [0, 1, 2, 3, 4]  # 5 states
        self.goal_state = 4  # Goal is state 4
```

A linear 5-state environment simulates a simple motor sequence. The sparse reward structure tests the agent's capacity for delayed gratification.

```
class ActorCriticAgent:
    def __init__(self, num_states, num_actions, alpha_actor=0.1,
alpha_critic=0.1, gamma=0.9, serotonin_level=0.5):
        self.num_states = num_states
        self.num_actions = num_actions
        self.gamma = gamma
        self.alpha_actor = alpha_actor
        self.alpha_critic = alpha_critic

        # Initialize the critic (state-value function) and the actor
(policy)
        self.V = np.zeros(num_states)  # State value function V(s)
```

```
        self.policy = np.ones((num_states, num_actions)) / num_actions  # Action probabilities π(a|s)

        # Serotonin influences exploration, modulating the softmax temperature
        self.temperature = 1.0 - serotonin_level  # Higher serotonin -> less exploration
```

Policy is updated using a softmax over action preferences, scaled by serotonin modulated temperature.

```
def softmax(self, preferences):
        """
        Computes a softmax over action preferences.
        The temperature modulates exploration vs exploitation.
        """
        scaled_prefs = preferences / self.temperature  # Scale by temperature
        exps = np.exp(scaled_prefs - np.max(scaled_prefs))  # Stability trick for exp
        return exps / np.sum(exps)  # Softmax probabilities
```

Each episode models a behavioural trial with thalamus relaying actions to the environment. Low serotonin leads to greater exploitation and slower convergence. The agent eventually learns to move rightward consistently to reach the goal state.

```
for episode in range(num_episodes):
    state = env.reset()
    total_reward = 0

    for step in range(20):  # Limit the number of steps in an episode
        # Choose an action based on the current policy
        action = agent.choose_action(state)

        # Take the action and observe the new state, reward, and whether the episode is done
        next_state, reward, done = env.step(action)

        # Update the agent's policy and value function using TD error
        agent.update(state, action, reward, next_state)
```